

Reference Notes

Week 12: Introduction to Functions

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How can we break down complex problems into manageable pieces using functions? This connects to our course's enduring question about functional decomposition and creating reusable code components that make our programs more organized and powerful.

Prerequisites

You should be comfortable with:

- Variables and data types from Module 1
- Basic control flow (if statements and loops)
- Working with lists and strings

1.2 What You Already Know

You've been writing Python code that runs from top to bottom, using loops to repeat actions and conditionals to make decisions. You've seen built-in functions like `print()`, `len()`, and `input()`, and you know they do specific jobs when you call them.

1.3 What You'll Be Able to Do

After this week, you'll be able to create your own functions to organize code, avoid repetition, and build modular programs. You'll understand how functions receive input through parameters, return output values, and maintain their own separate workspace for variables.

2 VIDEO 1: Introduction to Functions

2.1 Understanding Functions

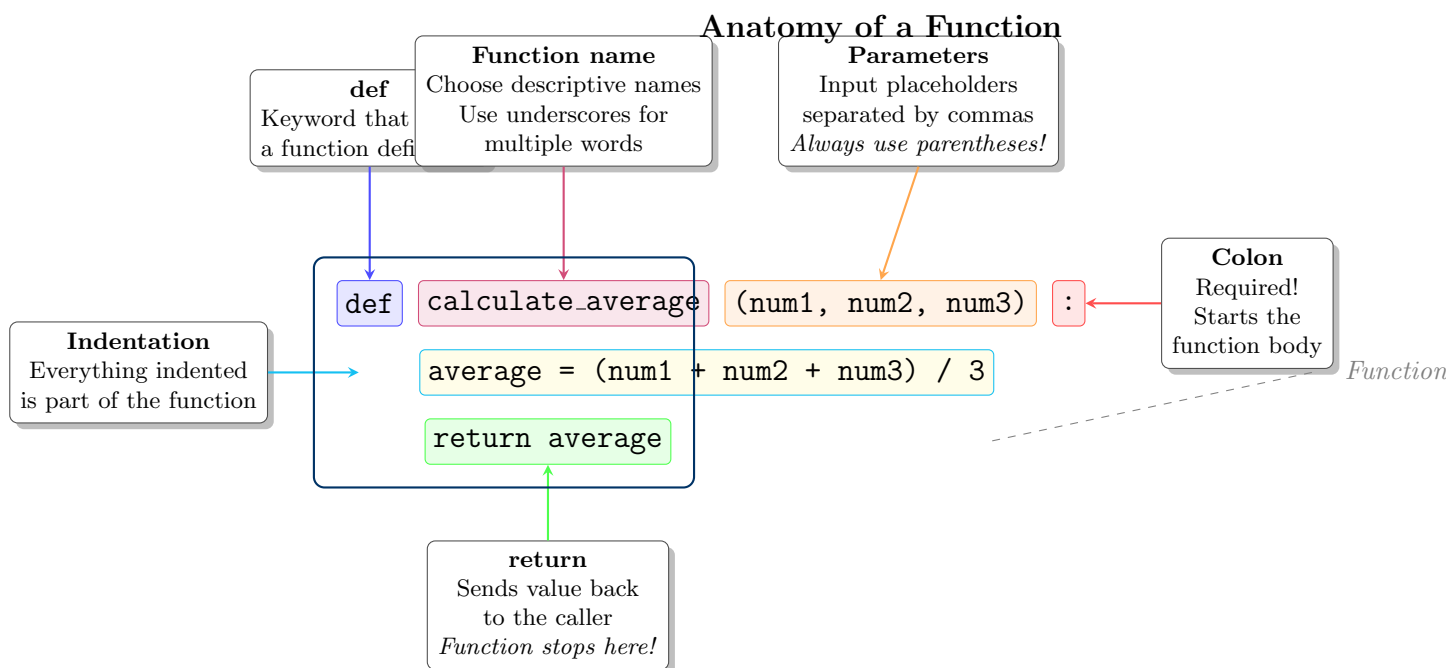
Functions are like recipe cards in programming - they contain a set of instructions to accomplish a specific task. Just as you might have a recipe for making chai that you can use whenever you want tea, functions let you package code that you can reuse whenever you need it. This makes your programs cleaner, easier to debug, and more powerful.

2.2 Your First Function

Let's create a simple function that greets someone. Notice how we define the function once, then can use it multiple times without rewriting the greeting code. The `def` keyword tells Python we're creating a new function.

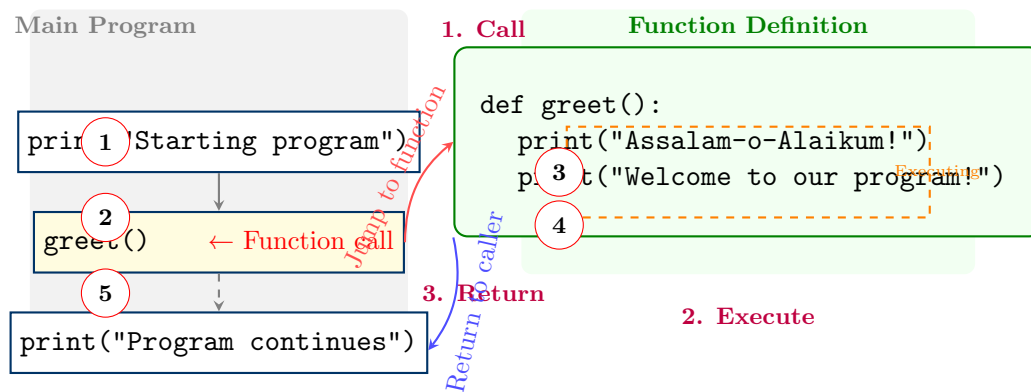
```
1 def greet():
2     print("Assalam-o-Alaikum!")
3     print("Welcome to our program!")
4
5 # Call the function three times
6 greet()
7 greet()
8 greet()
```

When Python sees `greet()`, it jumps to the function definition, runs the code inside, then returns to where it was called. This way, we write the greeting once but use it three times!



2.3 Visualizing Function Calls

To understand how functions work, let's visualize the flow of execution. When Python encounters a function call, it pauses the current code, jumps to the function, executes it, and then returns to continue where it left off.



This jumping behavior is what makes functions so powerful - they let us organize our code into logical chunks that can be called from anywhere in our program.

2.4 Functions with Different Tasks

Functions can do more than just print messages. They can perform calculations, process data, or make decisions. Let's see a function that calculates something useful for students.

```
1 def calculate_grade():
2     quiz = 85
3     homework = 90
4     exam = 78
5     average = (quiz + homework + exam) / 3
6     print(f"Your average is: {average:.1f}")
7
8 calculate_grade() # Calculates and displays the grade
```

This function encapsulates the grade calculation logic. Every time we call it, it performs the same calculation with the same values.

⚠ Common Error

Common Mistake: Forgetting the parentheses when calling a function. Writing `greet` instead of `greet()` won't call the function - it just refers to the function object without executing it.

👍 Quick Check

Write down what this code will print:

```
1 def say_hello():
2     print("Hello")
3
4 print("Start")
5 say_hello()
6 print("End")
```

3 VIDEO 2: Parameters and Arguments

3.1 Making Functions Flexible with Parameters

Our functions so far always do exactly the same thing. Parameters let us pass information to functions, making them flexible and reusable for different situations. Think of parameters as placeholders that get filled in when the function is called.

3.2 Adding Parameters to Functions

Let's upgrade our greeting function to greet different people by name. The parameter `name` acts like a variable that receives whatever value we pass when calling the function.

```
1 def greet_person(name):
2     print(f"Assalam-o-Alaikum, {name}!")
3     print(f"How are you today, {name}?")
4
5 greet_person("Ahmed")
6 greet_person("Fatima")
7 greet_person("Ali")
```

Each time we call `greet_person`, we provide a different name as an argument, and the function uses that specific name in its messages.

3.3 Multiple Parameters

Functions can accept multiple parameters to handle more complex tasks. Let's create a function that needs several pieces of information to calculate a student's weighted grade.

```
1 def calculate_weighted_grade(quiz, homework, exam):
2     # Different weights for each component
3     weighted = quiz * 0.2 + homework * 0.3 + exam * 0.5
4     print(f"Quiz: {quiz}, HW: {homework}, Exam: {exam}")
5     print(f"Weighted average: {weighted:.1f}")
6
7 calculate_weighted_grade(85, 90, 78)
8 calculate_weighted_grade(92, 88, 95)
```

The order matters! The first value goes to the first parameter, the second to the second parameter, and so on.

3.4 Visualizing Parameter Passing

When we call a function with arguments, Python creates a connection between the arguments we provide and the parameters in the function. This process is like filling in blanks in a form.

Diagram showing: - Function call: `calculate_weighted_grade(85, 90, 78)` - Arrows mapping `85`→`quiz`, `90`→`homework`, `78`→`exam` - Function definition with parameters receiving these values - Visual representation of parameter-argument matching

This matching process happens every time we call the function, allowing the same function to work with different values.

Tip

Choose descriptive parameter names that clearly indicate what kind of value is expected. Instead of `x` and `y`, use names like `student_name` or `test_score`.

Quick Check

What will this function print when called?

```
1 def introduce(name, age, city):
2     print(f"{name} is {age} years old from {city}")
3
4 introduce("Sara", 13, "Karachi")
```

4 VIDEO 3: Return Values

4.1 Functions That Give Back Results

So far, our functions have printed information, but sometimes we want functions to calculate something and give us the result to use elsewhere in our program. The `return` statement lets functions send values back to whoever called them.

4.2 Using Return Statements

Let's modify our grade calculator to return the result instead of printing it. This makes the function more versatile because we can use the returned value in different ways.

```
1 def calculate_average(num1, num2, num3):
2     average = (num1 + num2 + num3) / 3
3     return average
4
5 # Store the returned value in a variable
6 result = calculate_average(85, 90, 78)
7 print(f"The average is: {result}")
8
9 # Use the returned value in a calculation
10 final = calculate_average(92, 88, 95) + 5
11 print(f"With bonus: {final}")
```

The `return` statement sends the value back, and we can store it in a variable or use it directly in expressions.

4.3 Return vs Print

Understanding the difference between `return` and `print` is crucial. Let's see why they're different and when to use each one.

```
1 def add_with_print(a, b):
2     print(a + b) # Shows the result but returns None
3
4 def add_with_return(a, b):
5     return a + b # Sends the result back
6
7 x = add_with_print(3, 4) # Prints 7
```

```

8 y = add_with_return(3, 4) # Returns 7
9
10 print(f"x is: {x}") # x is: None
11 print(f"y is: {y}") # y is: 7

```

Functions with `print` display information but return `None`. Functions with `return` send back a value you can use.

⚠ Common Error

Common Mistake: Thinking `print()` returns the displayed value. Remember: `print()` always returns `None`. If you need to use a value later, you must `return` it, not `print` it.

4.4 Functions in Expressions

When functions return values, we can use them anywhere we'd use a regular value. This lets us build complex calculations from simple function pieces.

```

1 def circle_area(radius):
2     return 3.14159 * radius * radius
3
4 def rectangle_area(length, width):
5     return length * width
6
7 # Using returned values in calculations
8 total_area = circle_area(5) + rectangle_area(10, 6)
9 print(f"Total area: {total_area:.2f}")

```

Each function returns its calculated area, and we can add them together just like adding regular numbers.

💡 Rule

A function stops executing as soon as it hits a `return` statement. Any code after `return` won't run.

👉 Quick Check

Predict the output:

```

1 def mystery(x):
2     return x * 2
3     print("Done!") # Will this print?
4
5 result = mystery(5)
6 print(result)

```

5 VIDEO 4: Function Scope

5.1 Understanding Variable Scope

Variables created inside functions live in their own separate space called "local scope." They can't be accessed from outside the function, and they don't affect variables with the same name outside. This isolation helps prevent bugs and makes functions predictable.

5.2 Local vs Global Variables

Let's see how variables inside functions are separate from variables outside. Each function creates its own workspace for variables.

```

1 score = 100 # Global variable
2
3 def update_score():
4     score = 50 # Local variable (different from global)
5     print(f"Inside function: {score}")
6
7 update_score()
8 print(f"Outside function: {score}")
9
10 # Output:
11 # Inside function: 50
12 # Outside function: 100

```

The `score` inside the function is completely separate from the `score` outside. Changing one doesn't affect the other.

5.3 Visualizing Scope with Memory Diagrams

To truly understand scope, we need to visualize how Python manages different memory spaces for global and local variables. Think of each function as having its own room where it keeps its variables.

Memory diagram showing: - Global frame box containing: `score = 100` - Function frame box (`update_score`) containing: `score = 50` - Arrows showing function call creating new frame - Labels: "Global Space" and "Local Space (exists only during function call)" - Note showing local frame disappears after function returns

When the function ends, its local space and all variables in it disappear. The global space remains unchanged.

5.4 Passing Values Between Scopes

To share information between scopes, we use parameters and return values. This is the proper way for functions to communicate with the rest of your program.

```

1 def add_bonus(current_score, bonus):
2     # Create new score in local scope
3     new_score = current_score + bonus
4     return new_score # Send it back
5
6 # Global scope
7 player_score = 100
8 player_score = add_bonus(player_score, 25)
9 print(f"Updated score: {player_score}") # 125

```

We pass the global `player_score` as an argument, the function calculates a new value, and returns it for us to update the global variable.

Tip

Keep functions self-contained by passing in everything they need as parameters and returning their results. This makes functions easier to test and reuse.

👉 Quick Check

Draw a memory diagram for this code at the moment just before the function returns:

```
1 x = 10
2 def double(n):
3     result = n * 2
4     return result
5 y = double(x)
```

6 VIDEO 5: Summary

6.1 What We Learned This Week

This week, we discovered the power of functions - one of the most important concepts in programming! We learned how to create our own functions using `def`, make them flexible with parameters, get results back with `return`, and understand how function scope keeps our code organized and bug-free. You can now break down complex problems into smaller, manageable pieces, just like professional programmers do.

Functions transform the way we write code. Instead of long, repetitive programs, we can now create modular, reusable components. Each function is like a tool in your programming toolbox - you build it once and use it whenever you need that specific task done. This is a fundamental skill that you'll use in every program you write from now on!

★ Landmark Moment

You've unlocked the ability to create your own commands in Python! Just like `print()` and `len()` are built-in functions, you can now build custom functions for any task you can imagine. This is what makes you a true programmer - not just using tools others made, but creating your own!

7 Quick Reference

Quick Reference

Week 12 Essential Patterns:

Defining Functions:

- Pattern: `def function_name(parameters):`
- Common use: Creating reusable code blocks
- Common mistake: Forgetting the colon or indentation

Parameters and Arguments:

- Pattern: `def func(param1, param2):` called with `func(arg1, arg2)`
- Memory: Arguments are matched to parameters by position
- Debug tip: Print parameters at the start of function to verify values

Return Values:

- Pattern: `return value`
- Common use: Sending results back to the caller
- Common mistake: Confusing `return` with `print()`

Function Scope:

- Pattern: Local variables exist only inside functions
- Memory: Each function call creates a new local scope
- Debug tip: Draw scope boxes to track variable values

7.1 Reflection Questions

1. What was the hardest part about understanding functions this week? How did you work through it?
2. Think about a program you wrote earlier this semester. How could functions make it better?
3. When debugging a function, what strategy helped you most - adding print statements, drawing diagrams, or something else?
4. Celebrate one moment this week when functions suddenly "clicked" for you. What made it clear?
5. Can you think of a real-world system (like a restaurant or school) that works like functions with inputs and outputs?

Next week: We'll explore how functions can call other functions and build complete programs using multiple functions working together!